

React Fundamentals

From hand-writing DOM updates to declarative components. Set up React with Vite + TypeScript, build components, and bring them to life with state.

WHAT WE COVER

[Why React](#)[Components](#)[Vite + TS setup](#)[JSX](#)[Events](#)[useState](#)

On Day 4 you updated the page by hand — selecting elements and changing them one by one. That works for a few elements but quickly becomes a tangle as an app grows. React offers a better way: you describe what the UI should look like for the current data, and React updates the real DOM for you. Today we'll see why that matters, set up a real React project with Vite and TypeScript, build components, and make them interactive with state.

React Fundamentals

React lets you build UIs out of reusable **components** and describe them **declaratively** — you write what the screen should show for a given state, and React keeps the actual page in sync. With the TypeScript from Days 8–9, every component is type-safe from the start.

CLASS AGENDA • ~2 HOURS

0:00	Why React: the limits of manual DOM work	§1–2
0:20	Thinking in components	§3
0:35	Set up a project with Vite + TypeScript	§4
0:55	JSX and writing components	§5–6
1:20	Events and state with useState	§7–8
1:45	Conditional rendering, lists & wrap-up	§9–10

ON THIS HANDOUT

- | | |
|---|--------------------------------------|
| 01. The problem: manual DOM work | 06. Writing components |
| 02. What is React? | 07. Handling events |
| 03. Thinking in components | 08. State with useState |
| 04. Setup: Vite + React + TypeScript | 09. Conditional rendering |
| 05. JSX: HTML inside JavaScript | 10. Rendering lists with keys |

PART 1 • Why React

01 The problem: manual DOM work

Remember Day 4? To change the page you selected an element and updated it by hand. For a single counter that's fine:

```

<button id="btn">Clicked 0 times</button>

<script>
  let count = 0;
  const btn = document.querySelector("#btn");
  btn.addEventListener("click", () => {
    count++;
    btn.textContent = `Clicked ${count} times`; // you keep the DOM in sync by hand
  });
</script>

```

Now imagine dozens of elements that all depend on the same data — a list that can add, remove and reorder items, with counts and buttons that must all stay correct. You'd be writing endless `querySelector` + `textContent` updates, and it's very easy for one part of the screen to fall out of sync with another.

- ▶ You manually update every piece of the UI whenever data changes.
- ▶ As the app grows, keeping everything consistent becomes fragile and bug-prone.
- ▶ This imperative, step-by-step style simply doesn't scale.

02 What is React?

React is a library for building user interfaces out of **components**, using a **declarative** style. Instead of describing the steps to change the DOM, you describe what the UI should look like for the current data — and React figures out the DOM updates.

A useful mental model: **UI = f(state)**. Your screen is a function of your data. Change the data, and React re-renders the parts that need to change.

- ▶ **Declarative:** you say *what* to show, not the steps to update the DOM.
- ▶ **Component-based:** the UI is built from small, reusable pieces.
- ▶ **You change the data; React updates the screen** — no manual DOM juggling.

03 Thinking in components

A **component** is a reusable piece of UI — a button, a card, a navbar, a whole page. Components can contain other components, forming a tree. You build big interfaces by composing small ones (just like Tailwind utilities compose into layouts).

- ▶ A component is a function that returns some UI.
- ▶ Small components combine into bigger ones: `App` holds a `Header`, a `TaskList`, a `Footer` ...
- ▶ Reusing a component means writing it once and using it many times with different data.

PART 2 • Setup & Building Blocks

04 Setup: Vite + React + TypeScript

Vite is a fast tool that scaffolds and runs modern front-end projects. It uses npm and `package.json` — the same world as Day 7 — so this will feel familiar. Create a React + TypeScript app:

```
npm create vite@latest my-app -- --template react-ts

cd my-app
npm install      # install the dependencies
npm run dev      # start the dev server (opens http://localhost:5173)
```

TERMINAL

The important files inside `src/`:

- ▶ `main.tsx` — the entry point; it mounts your app into the HTML page.
- ▶ `App.tsx` — your root component, where you'll start building.
- ▶ Components live in `.tsx` files (`.ts` + JSX). The dev server reloads instantly as you save.

TRY IT YOURSELF

Run the three commands, open the dev server in your browser, then edit the heading in `App.tsx` and watch it update live.

05 JSX: HTML inside JavaScript

React components return **JSX** — HTML-like markup written right inside JavaScript. It looks like HTML with a few differences.

```
function App() {
  const name = "Abhishek";
  return (
    <div>
      <h1>Hello, {name}!</h1>
      <p>1 + 1 = {1 + 1}</p>
    </div>
  );
}
```

TSX

`{/* {} drops JavaScript into the markup */}`

- ▶ Wrap curly braces `{ }` around any JavaScript expression to insert it.
- ▶ Use `className` instead of `class` (class is a reserved word in JS).
- ▶ Return **one** root element (wrap siblings in a parent, or a `<>...</>` fragment).
- ▶ Tags must close — even self-closing ones: `<img />`, `<br />`.

06 Writing components

A component is a function whose name starts with a Capital letter and returns JSX. You use it like a custom HTML tag, and compose components together.

```
function Header() {
  return <h1>My First React App</h1>;
}

function App() {
  return (
    <div>
      <Header />      {/* use a component like a tag */}
      <p>Welcome!</p>
    </div>
  );
}

export default App;
```

- ▶ Component names are PascalCase (`Header` , `TaskCard`).
- ▶ Use a component as a self-closing tag: `<Header />` .
- ▶ Tomorrow (Day 11) you'll pass **props** into components to make them reusable with different data.

PART 3 · State & Interactivity

07 Handling events

React handles events with props like `onClick` and `onChange` — the same events from Day 4, now written in JSX and pointing at a function.

```
function Clicker() {
  function handleClick() {
    console.log("Button clicked!");
  }

  return <button onClick={handleClick}>Click me</button>;
}
```

- ▶ Pass a **function** to `onClick` — don't call it, just hand it over.
- ▶ Common ones: `onClick` , `onChange` , `onSubmit` — mirroring Day 4's events.

08 State with useState

Here's the big idea. A normal variable won't update the screen — when it changes, React doesn't know to re-render. **State** is special: change it, and React re-renders the component. You create state with the `useState` hook.

```
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState<number>(0); // typed state (Day 9 generics!)

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>+1</button>
      <button onClick={() => setCount(0)}>Reset</button>
    </div>
  );
}
```

- ▶ `useState(0)` returns a pair: the current value and a setter function.
- ▶ Always update via the setter (`setCount`) — never change the variable directly.
- ▶ Calling the setter tells React to re-render with the new value. UI = f(state) in action.
- ▶ `useState<number>(0)` types the state — exactly the generics idea from Day 9.

TRY IT YOURSELF

Build a `Counter` with +1, -1 and Reset buttons. Then add a second piece of state, `step`, and make +1 add `step` instead.

09 Conditional rendering

Because JSX lives in JavaScript, you show or hide UI with ordinary expressions.

```
function Status() {
  const [loggedIn, setLoggedIn] = useState<boolean>(false);

  return (
    <div>
      {loggedIn ? <p>Welcome back!</p> : <p>Please log in.</p>} /* ternary */
      {loggedIn && <button>Logout</button>} /* && shows only if
true */
      <button onClick={() => setLoggedIn(!loggedIn)}>Toggle</button>
    </div>
  );
}
```

- ▶ Use a `? :` ternary to choose between two pieces of UI.

- ▶ Use `&&` to render something only when a condition is true.

10 Rendering lists with keys

To render a list, `map` over an array and return JSX for each item — the same array-of-objects data you typed on Day 9. Each item needs a unique `key` so React can track it efficiently.

```
function TaskList() {  
  const tasks = [  
    { id: 1, title: "Learn React" },  
    { id: 2, title: "Build a component" },  
    { id: 3, title: "Ship it" },  
  ];  
  
  return (  
    <ul>  
      {tasks.map((task) => (  
        <li key={task.id}>{task.title}</li>  
      ))}  
    </ul>  
  );  
}
```

TSX

- ▶ `.map()` turns each data item into a piece of JSX.
- ▶ Give each element a stable, unique `key` (an id is perfect) so React updates the list correctly.

TRY IT YOURSELF

Render a list of your five favourite movies. Then add a `useState` counter that shows how many are in the list.

Practice Challenges

Combine today's pieces:

- ▶ Build a **toggle** button that switches text between "ON" and "OFF" with `useState`.
- ▶ Build a **like** button that shows a heart count and increments on click.
- ▶ Show a list of items and a message that says "List is empty" when there are none (conditional rendering).
- ▶ Make a tiny **tip calculator**: an input for the bill and buttons for 10% / 15% / 20% that show the tip.

Conclusion & what's next

You've made the jump from hand-updating the DOM to thinking in React: declarative, component-based UI. You set up a Vite + TypeScript project, wrote components with JSX, handled events, and made things interactive with `useState` — plus conditional rendering and lists.

Next class (Day 11): props — passing data from a parent component down to its children so components become truly reusable. We'll build some real UI and work more with hooks. From there: forms, routing, reusable components, state management, and a full e-commerce front-end.

QUICK RECAP

Manual DOM work doesn't scale · React is **declarative + component-based** ($UI = f(state)$) · scaffold with **Vite + TS** · components return **JSX** · **useState** holds data and re-renders on change · render lists with a **key**.